

MyPrecious

1. Analisi concettuale	2
1.1. Raffinamento dei requisiti	2
1.2. Diagramma UML delle classi	4
1.3. Specifiche delle classi	5
1.3.1. Specifica della classe Autore	5
Vincoli Esterni	5
[V.DataNascitaMinoreDataMorte]	5
1.3.2. Specifica della classe Opera	5
Vincoli Esterni	5
[V.AnnoRealizzazioneInPeriodoDiVitaAttore]	5
[V.AnnoRealizzazioneMinorePeriodoEsposizione]	5
1.3.3. Specifica della classe Partecipa	5
Operazioni ausiliarie	5
non_sovraposti(iniz1:Data, fin1: Data, iniz2:Data, fin2: Data): Boolean	5
Vincoli Esterni	5
1.3.4. Specifica della classe Esposizione	6
Vincoli Esterni	6
[V.DataInizMinoreDaraFine]	6
1.3.5. Specifica della classe Biglietto	6
Vincoli Esterni	6
[V.DataValiditaInAperturaEsposiizione]	6
1.3.6. Specifica della classe BigliettoStandard	6
calc_prezzo() : Reale >= 0	6
1.3.7. Specifica della classe BigliettoExtended	6
calc_prezzo() : Reale >= 0	6
1.4. Diagramma degli Use-Case	7
1.5. Specifica degli Use-Case	7
1.5.1. Specifiche dello use-case Organizzazione e gestione delle esposizioni	7
num_opere_non_esposte_dal_artista_in_periodo(at:Attore, d_iniz:DateTime, d_fine:DateTime): Intero >= 0	7
corrente_artisticha_popolare_nelle_mostreTemp_nel_periodo(d_iniz: Data, d_fine: Data) : CorrenteArtistica	7
1.5.2. Specifica dello use-case Statistiche Visite	8
num_biglExtended_vend_per_data(est: EsposizioneTemporanea, d:Date) : Intero >= 0	8
esposizioneTemp_con_max_num_vend_nel_periodo(d_iniz: DateTime, d_fin: DateTime) :	
EsposizioneTemporanea	8
num_esposizioniTemporanei_non_popolari_nel_periodo(d_iniz: Data, d_fin: Data): Intero >= 0	9
2. Design dell'applicazione in Python	10
2.1. Diagramma UML delle classi Ristrutturato	10
2.2. Specifiche dei tipi di dato	11
2.3. Specifiche delle Classi	11
2.3.1. Specifica della classe Autore	11
Vincoli Esterni	11
[V.DataNascitaMinoreDataMorte]	11
2.3.2. Specifica della classe Opera	11
Vincoli Esterni	11
[V.AnnoRealizzazioneInPeriodoDiVitaAttore]	11
[V.AnnoRealizzazioneMinorePeriodoEsposizione]	11
2.3.3. Specifica della classe Partecipa	11
Operazioni ausiliarie	11
non_sovraposti(	11

Vincoli Esterni	12
2.3.4. Specifica della classe Esposizione	12
Vincoli Esterni	12
[V.DataInizMinoreDaraFine]	12
2.3.5. Specifica della classe Biglietto	12
Vincoli Esterni	12
[V.DataValiditaInAperturaEsposiizione]	12
2.3.6. Specifica della classe BigliettoStandard	12
calc_prezzo() : Reale >= 0	12
2.3.7. Specifica della classe BigliettoExtended	13
calc_prezzo() : Reale >= 0	13
2.4. Specifiche degli Use-Case	13
2.4.1. Specifiche dello use-case Organizzazione e gestione delle esposizioni	13
num_opere_non_esposte_dal_artista_in_periodo(	13
corrente_artisticha_popolare_nelle_mostreTemp_nel_periodo(	14
2.4.2. Specifica dello use-case Statistiche Visite	15
num_biglExtended_vend_per_data(	15
esposizioneTemp_con_max_num_vend_nel_periodo(	16
num_esposizioniTemporanei_non_popolari_nel_periodo(	18
3. Ristrutturazione per BasiDati	19
3.1. Diagramma UML ristrutturato per bd	19
3.2. Traduzione diretta	20

1. Analisi concettuale

1.1. Raffinamento dei requisiti

Raffinamento dei requisiti

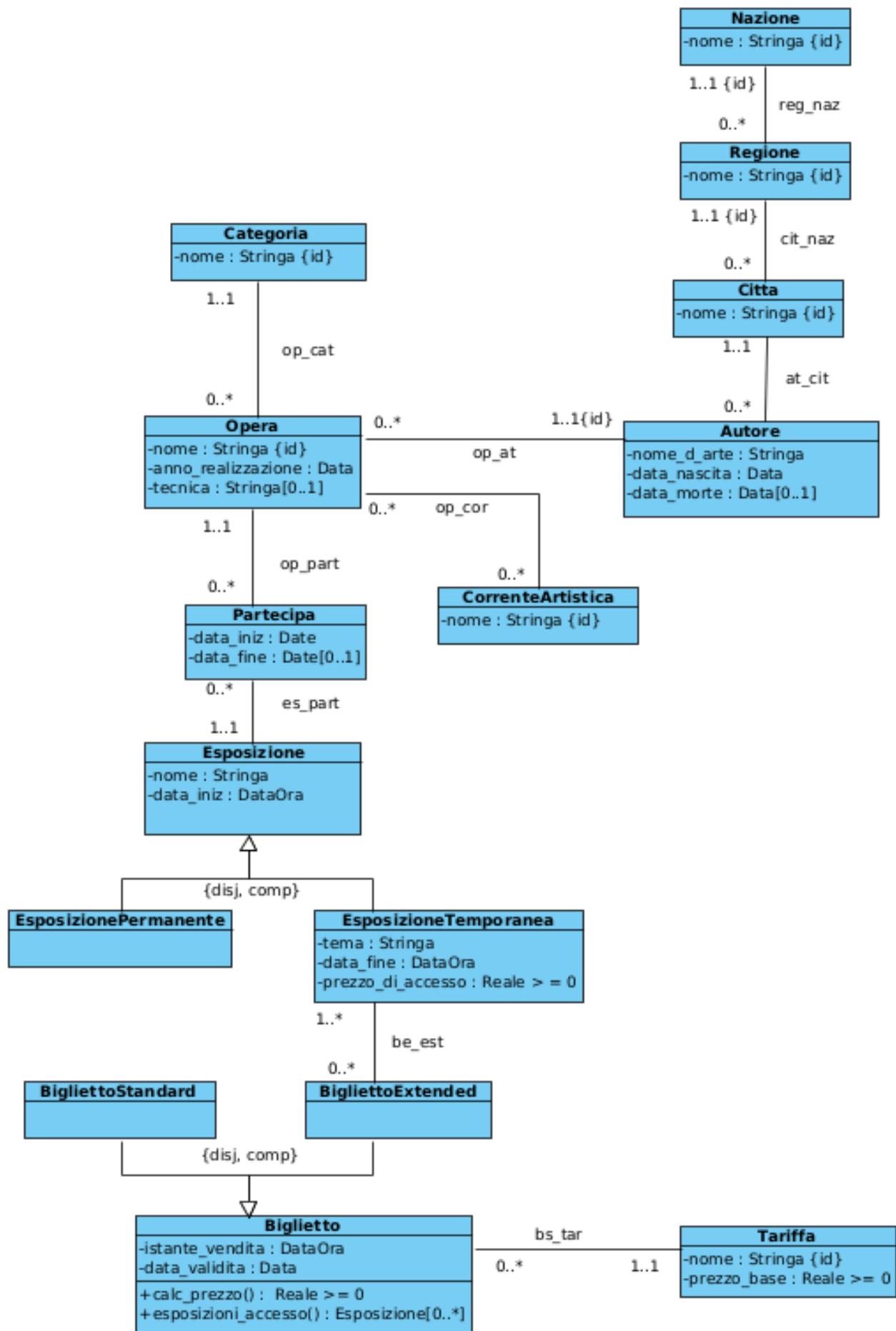
1. Requisiti relativi alle Opere d'arte
 - 1.1. nome
 - 1.2. anno realizzazione
 - 1.3. autore
 - 1.4. le correnti artistiche (piu di uno)
 - 1.5. categoria di arte (al massimo una)
 - 1.6. tecnica (se rilevante)
2. Requisiti relativi ai Autori
 - 2.1. nome d'arte
 - 2.2. data di nascita
 - 2.3. data di morte (se c'è)
 - 2.4. luogo di nascita
 - 2.5. opere che ha realizzato che sono di proprietà del museo
3. Requisiti relativi alle Esposizioni
 - 3.1. nome
 - 3.2. tipo (permanente o temporanea)
 - 3.2.1. esposizioni permanenti
 - 3.2.1.1. durata - sempre attive
 - 3.2.2. esposizioni temporanei
 - 3.2.2.1. durata - attivi per un periodo breve
 - 3.2.2.2. hanno un tema particolare (come un artista, un gruppo di artisti, una corrente artistica o un periodo storico)
 - 3.2.2.3. hanno un prezzo di accesso in piu
4. Requisiti relativi ai Biglietti
 - 4.1. tariffa scelta
 - 4.2. istante vendita
 - 4.3. data per la quale è valido (scelta dal visitatore)
 - 4.4. tipo (standard o extended access)
 - 4.4.1. biglietto standard
 - 4.4.1.1. accesso solo alle esposizioni permanenti
 - 4.4.1.2. prezzo uguale alla tariffa scelta
 - 4.4.2. biglietto Extended access
 - 4.4.2.1. accesso a tutte le esposizioni permanenti e alle esposizioni temporanei scelti
 - 4.4.2.2. prezzo uguale a prezzo base della tariffa scelta piu i prezzi di accesso alle esposizioni temporanei scelte
5. Requisiti relativi alle Tariffe
 - 5.1. nome
 - 5.2. prezzo base

Requisiti che riguardano le funzionalità

- 999.1. Responsabili del patrimonio artistico possono inserire e rimuovere di opere d'arte
- 999.2. Responsabili del patrimonio artistico possono aggiungere un artista e modificare delle sue informazioni;
- 999.3. Responsabili del patrimonio artistico possono creare delle esposizioni, con le opere relative;
- 999.4. Personale del management del museo possono fare la registrazione della vendita di uno o più biglietti di una certa tipologia e validi per una certa data.

999.5. Personale del management del museo possono fare il calcolo degli introiti totali per una certa esposizione temporanea, in un dato periodo.

1.2. Diagramma UML delle classi



1.3. Specifiche delle classi

1.3.1. Specifica della classe Autore

Vincoli Esterni

[V.DataNascitaMinoreDataMorte]

Per ogni at: Autore deve essere vero che:

- at.data_nascita < at.data_morte

oppure

- at.data_morte non è valorizzato.

1.3.2. Specifica della classe Opera

Vincoli Esterni

[V.AnnoRealizzazioneInPeriodoDiVitaAttore]

Per ogni op: Opera tale che (op, at) : op_at deve essere vero che:

- at.data_nascita < op.anno_realizzazione <= at.data_morte

oppure

- at.data_nascita < op.anno_realizzazione e at.data_morte non è valorizzato.

[V.AnnoRealizzazioneMinorePeriodoEsposizione]

Per ogni op: Opera tale che (op, part): op_part deve essere vero che:

- op.anno_realizzazione < part.data_iniz.

1.3.3. Specifica della classe Partecipa

Operazioni ausiliarie

non_sovraposti(iniz1:Data, fin1: Data, iniz2:Data, fin2: Data): Boolean

precondizioni:

iniz1 <= fin1 e iniz2 <= fin2

postcondizioni:

result = True se e solo se

fin1 < iniz2 or fin2 < iniz1

Vincoli Esterni

[V.PeriodiEsposizioneNonSovraposti]

Per ogni part: Partecipa

per ogni coppia di valori distinti

iniz1 e iniz2 di part.data_iniz e fin1 e fin2 di part.data_fine
deve essere vero che

non_sovraposti(
part.iniz1: Data,
part.fin1: Data,

part.iniz2: Data,
part.fin2: Data) = True

1.3.4. Specifica della classe Esposizione

Vincoli Esterni

[V.DataInizMinoreDataFine]

Per ogni est: EsposizioneTemporanea deve essere vero che:

- est.data_inizio < est.data_fine.

1.3.5. Specifica della classe Biglietto

Vincoli Esterni

[V.DataValiditaInAperturaEsposizione]

Per ogni bs: BigliettoStandard tale che (bs, esp): bs_esp dove esp: EsposizionePermanente deve essere vero che:

- bs.data_validita >= esp.data_iniz

Per ogni be: BigliettoExtended tale che (be, est): be_est dove est: EsposizioneTemporanea deve essere vero che:

- est.data_iniz <= be.data_validita <= est.data_fine.

1.3.6. Specifica della classe BigliettoStandard

calc_prezzo() : Reale >= 0

precondizioni: nessuna

postcondizioni:

- operazione non modifica i dati e link, (operazione non fa slide-effect)
- viene restituito il valore "result" definito come segue
 - sia tar: Tariffa tale che (this, tar): bs_tar
 - sia Pb = tar.prezzo_base
 - result = Pb

1.3.7. Specifica della classe BigliettoExtended

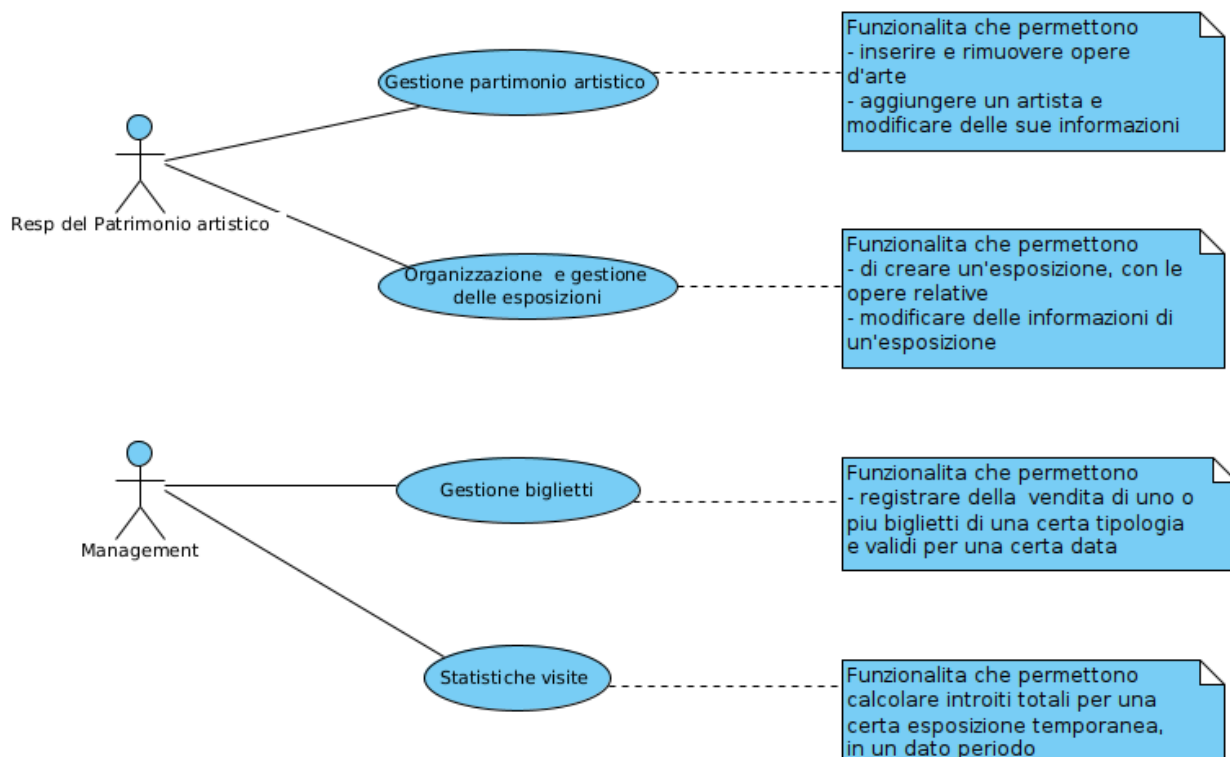
calc_prezzo() : Reale >= 0

precondizioni: nessuna

postcondizioni:

- operazione non modifica i dati e link, (operazione non fa slide-effect)
- viene restituito il valore "result" definito come segue
 - sia E insieme dei est: EsposizioneTemporanea tali che (this, est) : be_est
 - sia Sum è la somma di tutti i valori est.prezzo_accesso per ogni est in E
 - sia tar: Tariffa tale che (this, tar): bs_tar
 - sia Pb = tar.prezzo_base
 - result = Pb + Sum

1.4. Diagramma degli Use-Case



1.5. Specifica degli Use-Case

1.5.1. Specifiche dello use-case Organizzazione e gestione delle esposizioni

Dato un artista e un periodo di tempo, calcolare le opere realizzate dall'artista che non sono mai state esposte nel periodo dato.

`num_opere_non_esposte_dal_artista_in_periodo(at:Attore, d_iniz:DateTime, d_fine:DateTime): Intero >= 0`

precondizioni: nessuna

postcondizioni:

- l'operazione non modifica livello estenzionale dei oggetti
 - viene restituito il result definito come segue:
 - sia O l'insieme dei op : Opera tali che $(op, at): op_at$
 - sia O_n l'insieme di op in O per quali non esiste nessun link $(op, part): op_part$ dove $part$: Partecipa e $d_iniz \geq part.data_iniz$ e $d_fine \leq data_fine$
- `result = |O_n|`

nuova

Determinare, per un intervallo di tempo dato, le correnti artistiche che hanno avuto il maggior numero di opere esposte in mostre temporanee organizzate nel periodo indicato

`corrente_artistica_popolare_nelle_mostreTemp_nel_periodo( d_iniz: Data, d_fine: Data) : CorrenteArtistica`

precondizioni:

- esiste almeno un cor: CorrenteArtistica tale (op, cor): op_cor, op: Opera
- d_iniz < d_fine

postcondizioni:

- operazione non modifica il livello estenzionale degli oggetti
 - viene restituito "result" definito come segue:
 - Sia **ET** l'insieme di est: EsposizioneTemporanea tali che:
 - est.data_iniz >= d_iniz e est.data_fine <= d_fine
 - Sia **P** l'insieme di part: Partecipa tali che:
 - (est, part): es_part, est in ET
 - par.data_iniz >= d_iniz e part.data_fine <= d_fine
 - Sia **O** l'insieme di op: Opera tali che (op, part): p_part, part in P
 - Sia **CA** l'insieme di cor: CorrenteArtistica tali che (op, cor): op_cor, op in O
 - Sia **N** l'insieme delle coppie (cor, n) tali che
 - cor in CA
 - n = la cardinalità dell'insieme dei link op_cor ai quali partecipa cor
 - Sia (cor_max, n_max) l'elemento (cor, n) di N con il massimo n
- result: cor_max

1.5.2. Specifica dello use-case Statistiche Visite

Data un'esposizione temporanea e una data, calcolare il numero di biglietti Extended access venduti per tale esposizione e validi per tale data.

num_biglExtended_vend_per_data(est: EsposizioneTemporanea, d:Date) : Intero >= 0

precondizioni:

- est.data_iniz <= d <= est.data_fin

postcondizioni:

- l'operazione non modifica il livello estensionale degli oggetti
- viene restituito "result" definito come segue:
 - sia B insieme dei be: BigliettoExtended tali che:
 - be.data_validita = d e (be, est): be_est

result = |B|

Dato un periodo di tempo, calcolare le esposizioni temporanee per la quale è stato venduto il più alto numero di biglietti Extended access nel periodo.

esposizioneTemp_con_max_num_vend_nel_periodo(d_iniz: DateTime, d_fin: DateTime) : EsposizioneTemporanea

precondizioni:

- esiste almeno un link (est, be): be_est, est: EsposizioneTemporanea, be: BigliettoExtended e d_iniz <= be.istante_vendita <= d_fin
- d_iniz < d_fine

postcondizioni:

- operazione non modifica i dati e link
- viene restituito il result definito come segue:
 - Sia B insieme dei be: BigliettoExtended tali che:
 - d_iniz <= be.data_validita <= d_fin
 - Sia E insieme dei est: EsposizioneTemporanea tali che:
 - (be, est): be_est con be in B;
 - Sia N l'insieme delle coppie (est, n) tali che
 - est in E
 - n = la cardinalità dell'insieme dei link be_est ai quali partecipa est
 - (oppure) n = il numero di link be_est ai quali partecipa est

- Sia (est_max, n_max) l'elemento (est, n) di N con il massimo n

$result = est_max$

nuova

Dato un intervallo di date, calcolare le mostre temporanee che non sono mai state scelte in biglietti extended access all'interno di quelle date.

$num_esposizioniTemporanei_non_popolari_nel_periodo(d_iniz: Data, d_fin: Data): Intero$
 ≥ 0

precondizioni:

- $d_iniz < d_fin$

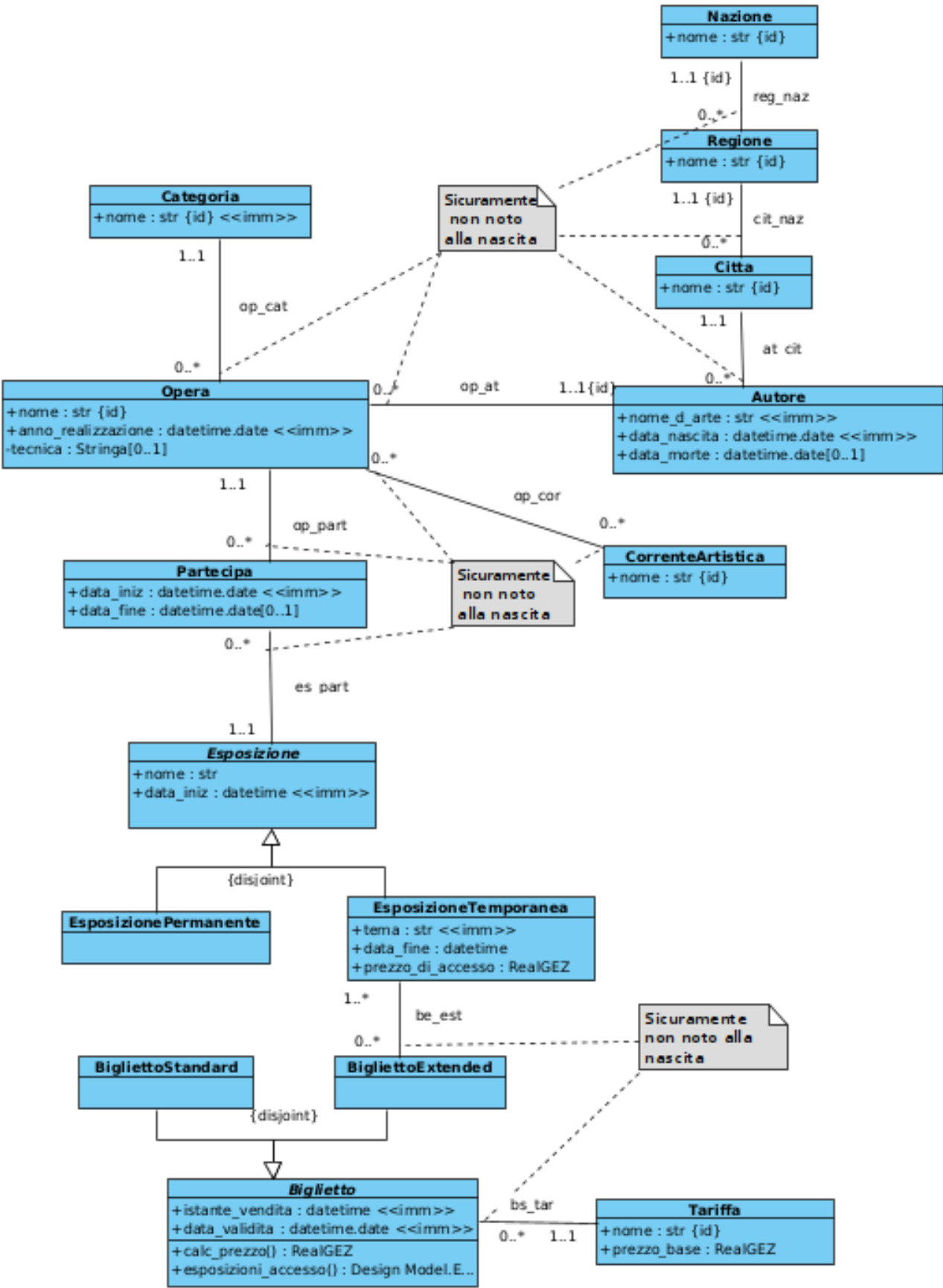
postcondizioni:

- operazione non modifica livello estenzionale dei oggetti
- viene restituito "result" definito come segue:
 - Sia E l'insieme di est : $EsposizioneTemporanea$ tali che
 $est.data_iniz \geq d_iniz$ e $est.data_fine \leq d_fine$
 - Sia En l'insieme di est in E tali che non esiste link (be, est) : be_est , be : $BigliettoExtended$

$result = |En|$

2. Design dell'applicazione in Python

2.1. Diagramma UML delle classi Ristrutturato



2.2. Specifiche dei tipi di dato

Tipo RealGEZ : float ≥ 0

Tipo IntGEZ : integer ≥ 0

2.3. Specifiche delle Classi

2.3.1. Specifica della classe Autore

Vincoli Esterni

[V.DataNascitaMinoreDataMorte]

Per ogni at: Autore deve essere vero che:

- at.data_nascita < at.data_morte

oppure

- at.data_morte non è valorizzato.

2.3.2. Specifica della classe Opera

Vincoli Esterni

[V.AnnoRealizzazioneInPeriodoDiVitaAttore]

Per ogni op: Opera tale che (op, at) : op_at deve essere vero che:

- at.data_nascita < op.anno_realizzazione $\leq$ at.data_morte

oppure

- at.data_nascita < op.anno_realizzazione e at.data_morte non è valorizzato.

[V.AnnoRealizzazioneMinorePeriodoEsposizione]

Per ogni op: Opera tale che (op, part): op_part deve essere vero che:

- op.anno_realizzazione < part.data_iniz.

2.3.3. Specifica della classe Partecipa

Operazioni ausiliarie

non_sovraposti(

iniz1: datetime.date,

fin1: datetime.date,

iniz2: datetime.date,

fin2: datetime.date

): bool

algoritmo:

se iniz1 > fin1 or iniz2 > fin2

ERRORE

```

se  fin1 < iniz2 or fin2 < iniz1
    return True
altrimenti
    return False

```

Vincoli Esterni

[V.PeriodiEsposizioneNonSovraposti]

Per ogni part: Partecipa

per ogni coppia di valori distinti

iniz1 e iniz2 di part.data_iniz e fin1 e fin2 di part.data_fine

deve essere vero che

```

non_sovraposti(
    part.iniz1: Data,
    part.fin1: Data,
    part.iniz2: Data,
    part.fin2: Data) = True

```

2.3.4. Specifica della classe Esposizione

Vincoli Esterni

[V.DataInizMinoreDataFine]

Per ogni est: EsposizioneTemporanea deve essere vero che:

- est.data_inizio < est.data_fine.

2.3.5. Specifica della classe Biglietto

Vincoli Esterni

[V.DataValiditaInAperturaEsposizione]

Per ogni bs: BigliettoStandard tale che (bs, esp): bs_esp dove esp: EsposizionePermanente deve essere vero che:

- bs.data_validita >= esp.data_iniz

Per ogni be: BigliettoExtended tale che (be, est): be_est dove est: EsposizioneTemporanea deve essere vero che:

- est.data_iniz <= be.data_validita <= est.data_fine.

2.3.6. Specifica della classe BigliettoStandard

calc_prezzo() : Reale >= 0

precondizioni: nessuna

postcondizioni:

- operazione non modifica i dati e link, (operazione non fa slide-effect)
- viene restituito il valore "result" definito come segue
 - sia tar: Tariffa tale che (this, tar): bs_tar
 - sia Pb = tar.prezzo_base

○ `result = Pb`

algoritmo:

```
link = this.bs_tar
prezzo = link.tariffa.prezzo_base
return prezzo
```

2.3.7. Specifica della classe BigliettoExtended

calc_prezzo() : Reale >= 0

precondizioni: nessuna

postcondizioni:

- operazione non modifica i dati e link, (operazione non fa slide-effect)
- viene restituito il valore "result" definito come segue
 - sia E insieme dei est: EsposizioneTemporanea tali che (this, est) : be_est
 - sia Sum è la somma di tutti i valori est.prezzo_accesso per ogni est in E
 - sia tar: Tariffa tale che (this, tar): bs_tar
 - sia Pb = tar.prezzo_base
 - `result = Pb + Sum`

algoritmo:

```
E = { }
sum = 0
per ogni link in this.be_est
    sum = sum + link.esposizioneTemporanea.prezzo_accesso

link = this.bs_tar
prezzo = link.tariffa.prezzo_base

return prezzo + sum
```

2.4. Specifiche degli Use-Case

2.4.1. Specifiche dello use-case Organizzazione e gestione delle esposizioni

Dato un artista e un periodo di tempo, calcolare le opere realizzate dall'artista che non sono mai state esposte nel periodo dato.

num_opere_non_esposte_dal_artista_in_periodo(

at: Autore,
d_iniz: datetime,
d_fine: datetime
): IntGEZ

precondizioni: nessuna

postcondizioni:

- l'operazione non modifica livello estensionale dei oggetti
- viene restituito il result definito come segue:
 - sia O l'insieme dei op: Opera tali che (op,at): op_at
 - sia O_n l'insieme di op in O per quali non esiste nessun link (op,part): op_part dove part: Partecipa e $d_{iniz} \geq part.data_{iniz}$ e $d_{fine} \leq data_{fine}$
- `result = |O_n|`

algoritmo:

```
# tutte le opere del autore
O = { }
per ogni link in at.op_at:
    O.add(link.opera)

se |O| == 0:
    return 0

O_esposte = { }

per ogni op in O:
    per ogni link in op.op_part:
        part = link.partecipa
        se d_iniz >= part.data_inizio and f_fine <= part.data_fine:
            O_esposte.add(op)

return O - O_esposte
```

nuova

Determinare, per un intervallo di tempo dato, le correnti artistiche che hanno avuto il maggior numero di opere esposte in mostre temporanee organizzate nel periodo indicato

corrente_artistica_popolare_nelle_mostreTemp_nel_periodo(

d_iniz: datetime.date,
d_fine: datetime.date
) : CorrenteArtistica

precondizioni:

- esiste almeno un cor: CorrenteArtistica tale (op, cor): op_cor, op: Opera

postcondizioni:

- operazione non modifica il livello estenzionale degli oggetti
- viene restituito "result" definito come segue:
 - Sia **E_p** l'insieme di est: EsposizioneTemporanea tali che:
 - est.data_iniz >= d_iniz e est.data_fine <= d_fine
 - Sia **P** l'insieme di part: Partecipa tali che:
 - (est, part): es_part, est in E_p
 - par.data_iniz >= d_iniz e part.data_fine <= d_fine
 - Sia **O** l'insieme di op: Opera tali che (op, part): p_part, part in P
 - Sia **CA** l'insieme di cor: CorrenteArtistica tali che (op, cor): op_cor, op in O
 - Sia **N** l'insieme delle coppie (cor, n) tali che
 - cor in CA
 - n = la cardinalità dell'insieme dei link op_cor ai quali partecipa cor
 - Sia (cor_max, n_max) l'elemento (cor, n) di N con il massimo n
- result: cor_max

algoritmo:

```
se d_iniz > d_fine:
    return Errore
```

E = insieme di tutti oggetti EsposizioneTemporanea

N : dict[CorrenteArtistica, IntGez] = { }

per ogni *est* in **E**:

se *est.data_iniz.date()* $\geq$ *d_iniz* and *est.data_fine.date()* $\leq$ *d_fine*:

per ogni *link_es_part* in **est.es_part**:

part = *link_es_part.artecipa*

se *part.data_fine* è valorizzato:

condizione = *part.data_iniz* $\geq$ *d_iniz* and *part.data_fine* $\leq$ *d_fine*:

altrimenti:

condizione = *part.data_iniz* $\geq$ *d_iniz* and *d_iniz* $\leq$ *d_fine*

se *condizione* == True:

per ogni *link_op_part* in **part.op_part**:

op = *link_po_part.opera*

per ogni *link_op_cor* in **op.op_cor**:

cor = *link_op_cor.correnteArtistica*

se *cor* in **N**:

N[cor] = *N[cor]* + 1

altrimenti:

N[cor] = 1

se **N** è vuoto:

return Errore

max_cor = None

max_op = None

primo = True

per ogni *k, v* in **N**:

se *primo* == True:

max_cor = *k*

max_op = *v*

primo = False

altrimenti:

se *v* > *max_op*:

max_cor = *k*

max_op = *v*

return *max_cor*

2.4.2. Specifica dello use-case Statistiche Visite

Data un'esposizione temporanea e una data, calcolare il numero di biglietti Extended access venduti per tale esposizione e validi per tale data.

num_biglExtended_vend_per_data(

est: EsposizioneTemporanea,

d:datetime.date

) : IntGEZ

precondizioni:

- *est.data_iniz* $\leq$ *d* $\leq$ *est.data_fin*

postcondizioni:

- l'operazione non modifica il livello estensionale degli oggetti
- viene restituito "result" definito come segue:
 - sia B insieme dei be: BigliettoExtended tali che:
be.data_validita = d e (be, est): be_est
 - result = |B|

algoritmo:

```

se est.data_iniz > d or est.data_fine < d:
    return Errore

B = { }
per ogni link in est.be_est:
    se link.bigliettoExtended.data_validita == d:
        B.add(link.bigliettoExtended)
return len(B)

```

Dato un periodo di tempo, calcolare le esposizioni temporanee per la quale è stato venduto il più alto numero di biglietti Extended access nel periodo.

esposizioneTemp_con_max_num_vend_nel_periodo(

d_iniz: datetime,

d_fin: datetime

) : EsposizioneTemporanea

precondizioni:

- esiste almeno un link (est, be): be_est , est: EsposizioneTemporanea, be: BigliettoExtended e
d_iniz <= be.istante_vendita <= d_fin

postcondizioni:

- operazione non modifica i dati e link
- viene restituito il result definito come segue:
 - Sia B insieme dei be: BigliettoExtended tali che:
 - d_iniz <= be.data_validita <= d_fin
 - Sia E insieme dei est: EsposizioneTemporanea tali che:
 - (be, est): be_est con be in B;
 - Sia N l'insieme delle coppie (est, n) tali che
 - est in E
 - n = la cardinalità dell'insieme dei link be_est ai quali partecipa est
 - (oppure) n = il numero di link be_est ai quali partecipa est
 - Sia (est_max, n_max) l'elemento (est, n) di N con il massimo n
 - result = est_max

algoritmo:

versione 1

```

se d_inizi > d_fin:
    return Errore

```

biglietti_ext = insieme di tutti oggetti BigliettiExtended

B insieme dei BigliettiExtended che erano venduti per il periodo che ci interessa

B = { }

per ogni be in **biglietti_ext**:

```

    se d_iniz <= be.data_validita and d_fin >= be.data_validita:
        B.add(be)

```

se len(B) == 0


```
return Errore
```

```
N : dict[EsposizioneTemporanea, IntGez] = { }
```

```
per ogni be in B:
```

```
    per ogni link in be.be_est:
```

```
        esp = link.EsposizioneTemporanea
```

```
        se esp not in N:
```

```
            N[esp] = 1
```

```
        altrimenti:
```

```
            N[esp] += 1
```

```
max_est = None
```

```
max_vend = None
```

```
primo = True
```

```
per ogni k, v in N:
```

```
    se primo == True:
```

```
        max_est = k
```

```
        max_vend = v
```

```
        primo = False
```

```
    altrimenti:
```

```
        se v > max_vend:
```

```
            max_est = k
```

```
            max_vend = v
```

```
return max_est
```

versione 2

```
se d_iniz > d_fin:
```

```
    return Errore
```

```
E = insieme di tutti oggetti EsposizioneTemporanea
```

```
N : dict[EsposizioneTemporanea, IntGez] = { }
```

```
per ogni est in E:
```

```
    sum = 0
```

```
    # verifico se esposizione era aperta nel periodo dato
```

```
    d2 = d_iniz if d_iniz >= est.data_iniz.date() else est.data_iniz.date()
```

```
    d2 = d_fin if d_fin <= est.data_fine.date() else est.data_fine.date()
```

```
    se d1 <= d2:
```

```
        d = d1
```

```
        finche d <= d2:
```

```
            sum = sum + num_biglExtended_vend_per_data( est, d)
```

```
            d = d + 1 giorno
```

```
    N[est] = sum
```

```
se N è vuoto:
```

```
    return Errore
```

```
max_est = None
```

```
max_vend = None
```

```

primo = True

per ogni k, v in N:
    se primo == True:
        max_est = k
        max_vend = v
        primo = False
    altrimenti:
        se v > max_vend:
            max_est = k
            max_vend = v

return max_est

```

nuova

Dato un intervallo di date, calcolare le mostre temporanee che non sono mai state scelte in biglietti extended access all'interno di quelle date.

num_esposizioniTemporanei_non_popolari_nel_periodo(

d_iniz: datetima.date,
 d_fin: datetime.date
): IntGEZ

precondizioni:

- d_iniz < d_fin

postcondizioni:

- operazione non modifica livello estenzionale dei oggetti
 - viene restituito "result" definito come segue:
 - Sia E l'insieme di est: EsposizioneTemporanea tali che
 $est.data_iniz \geq d_iniz$ e $est.data_fine \leq d_fine$
 - Sia E_n l'insieme di est in E tali che non esiste link (be, est): be_est, be: BigliettoExtended
- result = | E_n |

algoritmo:

```

se d_iniz >= d_fine:
    return Errore

```

Est_tutti = insieme di tutti oggetti EsposizioneTemporanea

E = { }

per ogni est in **Est_tutti**:

```

    se est.data_iniz.date() >= d_iniz and est.data_fine.date() <= d_fine:
        E.add(est)

```

```

se len(E) == 0:
    return 0

```

En = { }

per ogni est in **E**:

scelta = False

per ogni link_be_est in est.be_est:

```

    be = link_be_est.bigliettoExtended
    se d_iniz <= be.data_validita <= d_fine:
        scelta = True

```

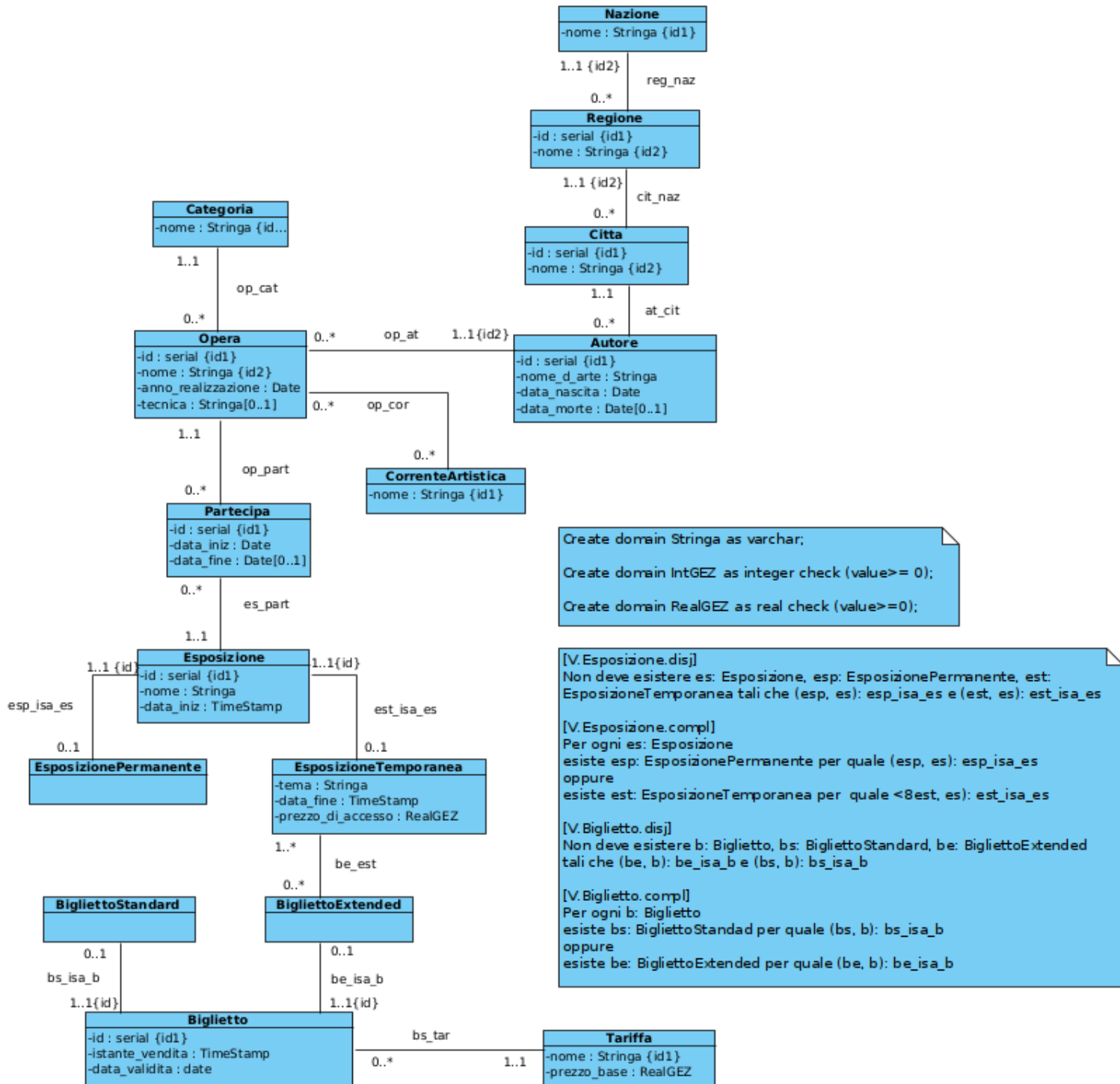
break

se scelta == False:
En.add(est)

return len(En)

3. Ristrutturazione per BasiDati

3.1. Diagramma UML ristrutturato per bd



3.2. Traduzione diretta

Create domain Stringa as varchar;

Create domain IntGEZ as integer check (value>= 0);

Create domain RealGEZ as real check (value>=0);

```
create table nazione (  
    nome stringa primary key  
);
```

```
create table regione (  
    id serial primary key,  
    nome stringa not null,  
    nazione stringa not null,  
    unique (nome, nazione),  
    foreign key (nazione)  
        references nazione(nome)  
);
```

```
create table citta (  
    id serial primary key,  
    nome stringa not null,  
    regione integer not null,  
    unique (nome, regione),  
    foreign key (regione)  
        references regione(id)  
);
```

```
create table autore (  
    id serial primary key,  
    nome_d_arte stringa not null,  
    data_nascita date not null,  
    data_morte date,  
    citta integer not null,  
    foreign key (citta)  
        references citta(id)  
);
```

```
create table categoria (  
    nome stringa primary key  
);
```

```
create table opera (  
    id serial primary key,  
    nome stringa not null,  
    anno_realizzazione date not null,  
    autore integer not null,  
    tecnica stringa,  
    unique (nome, autore),  
    foreign key (autore)  
        references autore(id)  
);
```

```
create table correnteArtistica (  
    nome stringa primary key  
);
```

```
create table op_cor (  
    opera integer not null,  
    correnteArt stringa not null,  
    primary key (opera, correnteArt),  
    foreign key (opera)  
        references opera(id),  
    foreign key (correnteArt)  
        references correnteArtistica(nome)  
);
```

```
create table esposizione (  
    id serial primary key,  
    nome stringa not null,  
    data_iniz timestamp not null  
);
```

```
create table esposizionePermanente (  
    esposizione integer primary key,  
    foreign key (esposizione)  
        references esposizione(id)  
);
```

```
create table esposizioneTemporanea (  
    esposizione integer primary key,  
    tema stringa not null,  
    data_fine timestamp not null,  
    prezzo_di_accesso RealGEZ not null,  
    foreign key (esposizione)  
        references esposizione(id)  
);
```

```
create table partecipa (  
    id serial primary key,  
    data_iniz date not null,  
    data_fine date,  
    opera integer not null,  
    esposizione integer not null,  
    foreign key (opera)  
        references opera(id),  
    foreign key (esposizione)  
        references esposizione(id)  
);
```

```
create table tariffa (  
    nome stringa primary key,  
    prezzo_base RealGEZ not null  
);
```

```
create table biglietto (  
    id serial primary key,  
    istante_vendita timestamp not null,  
    data_validita date not null,  
    tariffa stringa not null,  
    foreign key (tariffa)  
        references tariffa(nome)  
);
```

```
create table bigliettoStandart (  
    biglietto integer primary key,  
    foreign key (biglietto)  
        references biglietto(id)  
);
```

```
create table bigliettoExtended (  
    biglietto integer primary key,  
    foreign key (biglietto)  
        references biglietto(id)  
    -- v.inclusione BigliettoExtended(biglietto) occorre in be_est(bigliettoExt)  
);
```

```
create table be_est (  
    esposizioneTemp integer not null,  
    bigliettoExt integer not null,  
    foreign key (esposizioneTemp)  
        references esposizioneTemporanea(esposizione),  
    foreign key (bigliettoExt)  
        references bigliettoExtended(biglietto)  
);
```